

The Trade-Decision Brain

HFT Project • July 2026

What the system checks *before* it places a trade — modelled on how real HFT desks decide.

1. The big idea: a signal is not a trade

The FPGA (and its software twin, `strategy_sw.py`) produces a **signal** from order-book imbalance: “there are far more buyers than sellers right now, so lean long.” That signal is *necessary* but not *sufficient*. A real trading firm never fires an order on a raw signal alone — it runs the signal through a sequence of sanity checks, any one of which can **veto** the trade or **shrink** its size.

This document explains that sequence — the “brain” that sits between the raw signal and an actual order. Two design rules shaped it:

- **The fast core is untouched.** The imbalance signal in `strategy_sw.py` stays bit-for-bit identical to the FPGA (that equivalence is guarded by a test). All the logic here is a *software overlay* on top — exactly how real systems separate a fast signal engine from slower risk/execution checks.
- **Every rejection is counted.** The system logs *why* each signal was dropped, so you can watch the flow thin out from many signals down to few orders (the “funnel” in §3).

2. The pre-trade gauntlet (in order)

A signal must survive *every* gate below, top to bottom, to become an order. Code: `host/hft_logic.py`.

Gate 1 — Fair value (microprice)

Question: is the signal confirmed by the true fair price, not just the midpoint?

Plain version: don’t pay the sticker price — work out what the thing is actually worth right now. The naive “mid” is halfway between bid and ask, but if there is far more size resting on the bid, the real fair value is a little *higher* than the mid. That size-weighted fair value is the **microprice**:

$$\text{microprice} = \frac{P_{\text{bid}} \cdot S_{\text{ask}} + P_{\text{ask}} \cdot S_{\text{bid}}}{S_{\text{bid}} + S_{\text{ask}}}$$

We only **buy** if the microprice sits above the mid by at least a small margin (`min_edge_ticks`), and only **sell** if it sits below by that margin. If the microprice disagrees with the signal, the trade is vetoed.

Gate 2 — Edge versus cost

Question: is the expected profit bigger than the cost of trading?

Plain version: don’t drive across town to save a dollar on gas. Every trade pays a fee (modelled as `fee_bps`, basis points of the trade value). If the expected edge (how far the microprice is from the mid) does not clear the fee, skip the trade. This is what separates a *profitable* signal from a merely *correct* one.

Gate 3 — Market-condition gates

Question: is the market safe to trade into right now? Three quick checks:

- **Volatility** — stand down when short-term price swings spike (`max_vol_bps`). In fast markets you get “picked off” by faster players.
- **Liquidity** — require a minimum number of shares at the top of book (`min_book_size`). Thin books mean bad fills.
- **Staleness** — require a fresh quote (`max_quote_age_s`). Never act on an old price.

Gate 4 — Inventory skew

Question: does this trade help or hurt my current position?

Plain version: lean toward flat. Trades that *reduce* the position always pass. Trades that *add* to it are blocked once the position grows past a fraction of the per-symbol cap (`inventory_soft_frac`). This stops the book from piling up one-sided.

Gate 5 — Portfolio exposure & the kill switch

Question: across *all* symbols, am I within my risk budget — and have I lost too much today?
This gate consults the shared **portfolio** (§4):

- **Gross-exposure cap** — the order size is clamped to the dollar headroom left under the firm-wide gross-notional limit.
- **Kill switch** — if the day’s losses (or drawdown from the peak) breach the limit, *all trading stops*, on every symbol. This is the single most important safety brake a real desk has.

3. The funnel: many signals, few orders

Because every veto is counted, a run prints a funnel. A representative 7-symbol (Magnificent 7) simulation:

Stage	Count
Raw signals evaluated	202
vetoed — edge vs cost	139
vetoed — inventory skew	9
vetoed — liquidity	8
Orders passed	46

Reading it: of 202 raw signals, most (139) were not worth their cost, a few were blocked for risk/liquidity reasons, and 46 became orders. *That thinning is the whole point* — it is the difference between a naive strategy that trades on every twitch and a disciplined one that trades only when several independent conditions agree.

4. The portfolio: PnL, exposure, and the brake

Code: `host/portfolio.py`. One portfolio object is shared by all symbols and answers the questions a desk asks continuously:

- **Position** — signed shares held per symbol (+ long, – short).
- **Realized PnL** — profit booked when a trade reduces a position, using average-cost accounting.
- **Unrealized PnL** — open profit on the current position, marked to the latest price.
- **Net exposure** — long minus short (directional risk).
- **Gross exposure** — long plus short (total capital at work).
- **Kill switch** — trips on a daily-loss or drawdown breach.

5. An honest note on where the edge really is

If you run the simulation you will often see equity drift slightly *negative*. That is not a bug — it is the truth about a strategy that *crosses the spread* (a “taker”): after fees, a naive imbalance signal is roughly break-even to slightly losing. Real HFT profit comes from one of two places this project is honest about:

1. **Speed** — being first to react. That is *this project’s actual edge*: the FPGA computes the decision in **205 ns** versus microseconds in software. The decision *logic* here is a faithful model; the *advantage* is latency.
2. **Market-making** — posting two-sided quotes to *earn* the spread rather than pay it. That is a different strategy than the aggressive taker modelled here, and a natural future extension.

6. Where it lives in the code

File	Role
<code>rtl/strategy_imbalance.sv</code>	the raw signal, in hardware (FPGA)
<code>host/strategy_sw.py</code>	the raw signal, bit-identical in software
<code>host/hft_logic.py</code>	the pre-trade gauntlet (Gates 1–5)
<code>host/portfolio.py</code>	positions, PnL, exposure, kill switch
<code>host/live_feed.py</code>	glues quotes → signal → gauntlet → order
<code>host/tb_hft_logic.py</code>	unit tests for the whole overlay

Try it: `python host/live_feed.py -universe mag7 -simulate` (dry-run, no real orders — prints the funnel and portfolio at the end).